

Javascript

Theory

1. Promises

- **Definition:** A Promise represents the eventual completion or failure of an asynchronous operation.
- **States:**
 - Pending → initial state.
 - Fulfilled → operation completed successfully.
 - Rejected → operation failed.
- **Chaining:** .then() for success, .catch() for errors, .finally() for cleanup.
- **Example:**

```
new Promise((resolve, reject) => {  
  
  let success = true;  
  
  success ? resolve("Done") : reject("Error");  
  
});
```

2. Async/Await

- **Definition:** Syntactic sugar over Promises for cleaner asynchronous code.
- **Use case:** Avoids callback hell and makes async code look synchronous.
- **Example:**

```
async function fetchData() {  
  
  const res = await fetch("https://jsonplaceholder.typicode.com/todos");  
  
  return res.json();  
  
}
```

3. Promise Utilities

- **Promise.all()** → runs multiple promises in parallel, resolves when all succeed.
- **Promise.race()** → resolves/rejects as soon as one promise finishes.
- **Promise.any()** → resolves when any one succeeds.

4. Browser Objects

- **Window** → global object (alert, open, close, setTimeout, setInterval).
- **Screen** → info about display (width, height, availWidth, availHeight).
- **Location** → URL info (href, host, pathname, protocol).
- **History** → navigation (back(), forward(), go()).

5. LocalStorage

- **Definition:** Stores key-value pairs in the browser with no expiration.
- **Methods:**
 - `setItem(key, value)` → store.
 - `getItem(key)` → retrieve.
 - `removeItem(key)` → delete.
 - `clear()` → remove all.
- **Note:** Values must be strings → use `JSON.stringify()` and `JSON.parse()` for objects.

Coding Round Questions

- Promise basics → Explain Promise states with an example.
- Async/Await → Write a function using async/await to fetch data.
- Promise utilities → Show how `Promise.all()` works with multiple async tasks.
- LocalStorage → Store and retrieve an object using LocalStorage.
- Browser objects → Explain Window, Screen, Location, and History objects with examples.
- Error handling → Demonstrate error handling in async/await with try/catch.

⚡ Machine Coding Round Challenge

Problem: Todo List Manager with Async Fetch

Requirements:

- Fetch todos from <https://jsonplaceholder.typicode.com/todos>.
- Display first 10 todos in a list.
- Strike through incomplete todos.
- Store fetched todos in LocalStorage.
- Add a form to create a new todo → update both LocalStorage and UI.
- Use `Promise.all()` to simulate fetching user info and todos together.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Promise</title>
  </head>
  <body>
    <h2>Promise</h2>

    <ul id="posts"></ul>

    <script>
      // window.alert("Alert message");
      console.log(window.innerHeight);
      console.log(window.innerWidth);
      // window.open("http://google.com")

      // setTimeout(() => {

      //   window.close();
      // }, 10000);

      console.log(screen.width); // Screen width
      console.log(screen.height); // Screen height
      console.log(screen.availWidth); // Available width
      console.log(screen.availHeight); // Available height

      // history.back();
      // history.forward();
      // history.go(-2);

      console.log(location.href); // Full URL
      console.log(location.host); // Hostname + port
      console.log(location.pathname); // Path of URL
      console.log(location.protocol); // https: or http:

      // location.assign("https://google.com"); // Redirect
      // location.reload();

      localStorage.setItem("username", "FIIT");

      document.write(localStorage.getItem("username"));

      // localStorage.removeItem("username");

      // localStorage.clear();
```

```

let obj = {
  name: "Ganesh",
  degree: "LLB",
  age: 25
}
localStorage.setItem("obj", JSON.stringify(obj));

console.log(JSON.parse(localStorage.getItem("obj")));

async function fetchData() {
  return "Data fetched";
}

async function displayData() {
  const data = await fetchData();
  console.log(data);
}

displayData();

let count = 0;
const postContainer = document.getElementById("posts");

async function jsonPlaceHolderExample() {
  try {
    const result = await fetch(
      "https://jsonplaceholder.typicode.com/todos",
    );
    const data = await result.json();
    // postContainer.innerHTML = `<h2>${data.id}</h2>`
    // <h2>${data.title}</h2>`;
    data.slice(0, 10).forEach((ele) => {
      count++;
      const list = document.createElement("li");
      list.textContent = ele.title;

      if (!ele.completed) {
        list.style.textDecoration = "line-through";
      }
      postContainer.appendChild(list);
      // console.log(count);
    });
    // console.log(result);
    // console.log(data);
  } catch (error) {

```

```
        console.log(error);
    }
}

jsonPlaceHoldeExample();

function fetchDetails() {
    return new Promise((resolve) => {
        setTimeout(() => {
            resolve(1);
        }, 2000);
    });
}

Promise.all([fetchDatta(), fetchDetails()]).then((res) =>
    console.log(res),
);
</script>
</body>
</html>
```